

B. K. Birla College of Arts, Science & Commerce, Kalyan
Empowered Autonomous Status
T.Y.B.Sc. (Information Technology) 2026-27

Sr. No.	TOPICS	PAGE NO	DATE	SIGN
1	Pre-requisites of Game Programming			
2	Setup Game Window Framework and Initialize Pygame Environment			
3	Draw a Triangle Using Pygame			
4	Apply Texture to a Triangle Using Pygame			
5	Simulating Diffuse Lighting Using Pygame			
6	Simulating Specular Lighting Using Pygame			
7	Loading and Rendering Game Models Using Pygame			
8	Develop a 2D UFO Catching Game Using Python (Pygame)			
9	Develop a Roll-Ball Game Using Python (Pygame)			
10	Develop a Space Shooter Game Using Python (Pygame)			

Practical No. 1 : Pre-requisites of Game Programming

Aim: To study the basic requirements, tools, software, and concepts necessary for game development using Python.

Objectives

- Understand the fundamentals of game programming.
- Learn the software and hardware requirements for game development.
- Install and configure Python and Pygame.
- Create and execute a basic game window.

Requirements

Software Requirements

- Python (Version 3.x)
- Pygame
- Visual Studio Code / PyCharm Community Edition

Hardware Requirements

- Intel i3 / i5 Processor or higher
- Minimum 4 GB RAM
- 1 GB Free Disk Space
- Keyboard and Mouse
- Windows / Linux Operating System

Theory

Game Programming is the process of developing interactive computer games using programming languages and game development frameworks. It involves graphics rendering, animation, sound integration, collision detection, user input handling, and game logic implementation.

Components of a Game:

- Game Loop
- Graphics Rendering
- User Input Handling
- Collision Detection
- Sound Effects and Music
- Score Management
- Artificial Intelligence (Optional)

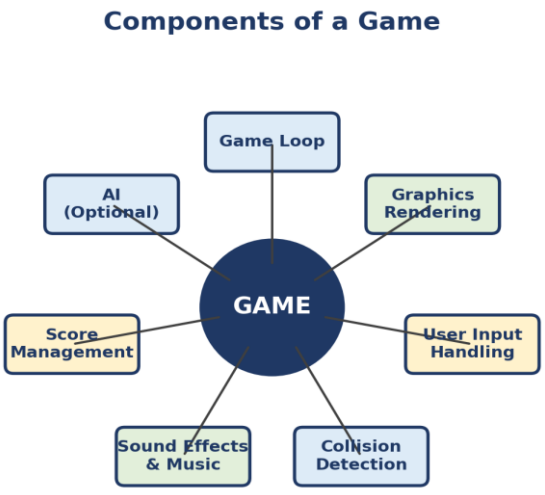


Fig. 1.1 — Components of a Game

Why Use Python for Game Development?

- Easy to learn and implement
- Large community support
- Suitable for 2D game development
- Availability of the Pygame library
- Rapid prototyping capability

Introduction to Pygame: Pygame is an open-source Python library used for creating 2D games. It provides modules for graphics, sound, event handling, and sprite management.

Procedure

Step 1: Install Python

Download and install Python from the official website, then verify the installation:

```
python --version
```

Step 2: Install Pygame

Open Command Prompt and execute:

```
pip install pygame
```

Verify installation:

```
pip show pygame
```

Step 3: Create a Simple Game Window

Create a file named "game_window.py":

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("My First Game Window")

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    screen.fill((0, 0, 0))
    pygame.display.update()

pygame.quit()
```

Sample Output

A black game window of size 800 x 600 pixels is displayed with the title “My First Game Window”.

Result: Successfully installed Python and Pygame, studied the prerequisites of game programming, and created a basic game window.

Practical No. 2 : Setup Game Window Framework and Initialize Pygame Environment

Aim: To create a game window framework and initialize the Pygame environment for game development.

Objectives

- Understand game initialization.
- Create a game window using Pygame.
- Initialize game resources.
- Implement the basic game loop.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code

Theory

In DirectX, the first step is to initialize the Direct3D Device and create a rendering window. In Python, the equivalent process is performed using the Pygame library.

Components Initialized:

- Pygame Engine
- Display Window
- Frame Rate Controller
- Event Handling System
- Game Loop

Game Loop Structure

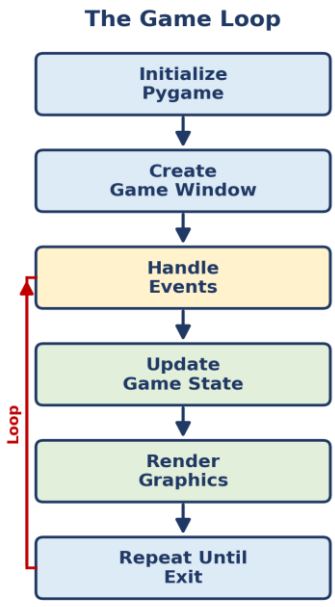


Fig. 2.1 — Structure of the Game Loop

Algorithm

1. Import Pygame module.
2. Initialize Pygame.
3. Create display window.
4. Set window title.
5. Create clock object.
6. Start game loop.
7. Handle user events.
8. Update display continuously.
9. Exit when user closes the window.

Program

```
import pygame

# Initialize Pygame
pygame.init()

# Window Settings
WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Game Window Framework")

# Clock Object
clock = pygame.time.Clock()

# Main Loop Variable
running = True
while running:
    # Event Handling
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Background Color
    screen.fill((30, 30, 30))

    # Display Update
    pygame.display.update()

    # Frame Rate
    clock.tick(60)

pygame.quit()
```

Explanation

- pygame.init() — Initializes all Pygame modules.
- pygame.display.set_mode() — Creates the game window.
- pygame.display.set_caption() — Sets the title of the game window.

- pygame.time.Clock() — Controls the frame rate.
- clock.tick(60) — Limits execution to 60 Frames Per Second (FPS).
- pygame.display.update() — Refreshes the display screen.

Sample Output

A window of size 800 x 600 appears with a dark background and remains active until closed.

Result: Successfully created a game window framework and initialized the Pygame environment for game development.

Practical No. 3 : Draw a Triangle Using Pygame

Aim: To draw a triangle on the game window using the Pygame graphics library.

Objectives

- Understand basic 2D graphics rendering.
- Learn coordinate-based drawing.
- Draw a triangle using vertices.
- Display graphical objects on the screen.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code

Theory

In Direct3D 11, a triangle is the fundamental primitive used for rendering graphics. Complex 3D objects are created by combining multiple triangles. In Pygame, a triangle can be drawn using the pygame.draw.polygon() function by specifying three vertices.

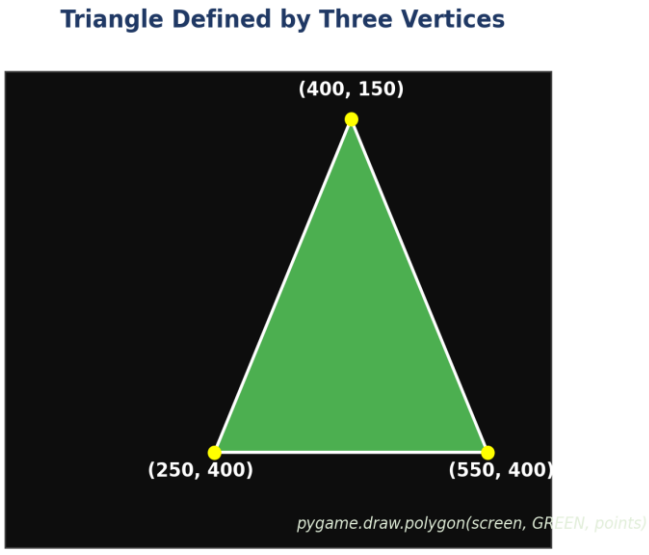


Fig. 3.1 — Triangle Represented by Three Vertices

Algorithm

1. Import Pygame library.
2. Initialize Pygame.
3. Create a game window.
4. Define three vertices of a triangle.
5. Draw the triangle using pygame.draw.polygon().
6. Update the display.
7. Close the window when the user exits.

Program

```
import pygame

# Initialize Pygame
pygame.init()

# Window Size
WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Triangle Drawing")

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Background Color
    screen.fill((0, 0, 0))

    # Triangle Coordinates
    points = [(400, 150), (250, 400), (550, 400)]

    # Draw Triangle
    pygame.draw.polygon(screen, (0, 255, 0), points)

    # Update Display
    pygame.display.update()

pygame.quit()
```

Explanation

pygame.draw.polygon(surface, color, points)

- surface — the screen object
- color — the RGB color of the shape
- points — the list of vertex coordinates, e.g. [(400,150), (250,400), (550,400)], which define the three vertices of the triangle.

Sample Output

A green triangle is displayed on a black background.

Result: Successfully created a game window and rendered a triangle using Pygame graphics functions.

Practical No. 4 : Apply Texture to a Triangle Using Pygame

Aim: To load an image texture and display it on a triangular game object using Pygame.

Objectives

- Understand the concept of textures in game development.
- Learn image loading and rendering.
- Apply texture effects to graphical objects.
- Display textured game elements.

Requirements

Software Requirements

- Python
- Pygame
- A PNG / JPG image file (texture)

Theory

A texture is an image applied to a graphical object to make it look realistic. In Direct3D, textures are mapped onto polygons using texture coordinates. True texture mapping on triangles is not directly supported in Pygame; therefore the concept is simulated by loading an image, rendering it inside a triangular region, and drawing a triangle boundary over the textured area.

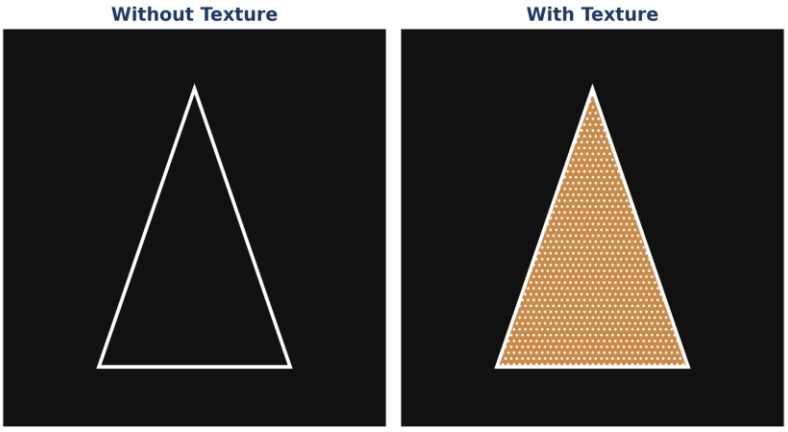


Fig. 4.1 — Triangle Without and With Texture

Algorithm

1. Import Pygame.
2. Initialize Pygame.
3. Create game window.
4. Load texture image.
5. Resize texture image.
6. Display texture on screen.
7. Draw triangle boundary over texture.
8. Refresh display continuously.

Program

Save an image named "texture.jpg" in the project folder.

```
import pygame

pygame.init()

WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Textured Triangle")

# Load Texture
texture = pygame.image.load("texture.jpg")
texture = pygame.transform.scale(texture, (300, 250))

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((0, 0, 0))

    # Display Texture
    screen.blit(texture, (250, 150))

    # Triangle Outline
    points = [(400, 150), (250, 400), (550, 400)]
    pygame.draw.polygon(screen, (255, 255, 255), points, 3)

    pygame.display.update()

pygame.quit()
```

Explanation

- pygame.image.load("texture.jpg") — loads an image from disk.
- pygame.transform.scale(texture, (300,250)) — resizes the image.
- screen.blit(texture, (250,150)) — draws the texture on the screen.
- pygame.draw.polygon() — draws the triangle outline.

Sample Output

The triangle appears with an image texture effect and a visible white boundary.

Result: Successfully loaded and displayed a texture image and demonstrated the concept of texture mapping on a triangular game object using Pygame.

Practical No. 5 : Simulating Diffuse Lighting Using Pygame

Aim: To simulate diffuse lighting effects on a game object using Python and Pygame.

Objectives

- Understand the concept of diffuse lighting.
- Simulate light intensity based on object orientation.
- Create realistic lighting effects in a 2D environment.
- Learn the basics of lighting used in game development.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code

Theory

Diffuse lighting is the most common lighting model used in computer graphics. It simulates how light reflects equally in all directions from a rough surface. In Direct3D 11, diffuse lighting is implemented using shaders, where intensity depends on the angle between the Surface Normal Vector (N) and the Light Direction Vector (L).

The diffuse lighting equation is:

$$Id = kd\ (N \cdot L)$$

- Id — Diffuse light intensity
- kd — Diffuse reflection coefficient
- N — Surface normal vector
- L — Light direction vector
- N . L — Dot product of the two vectors

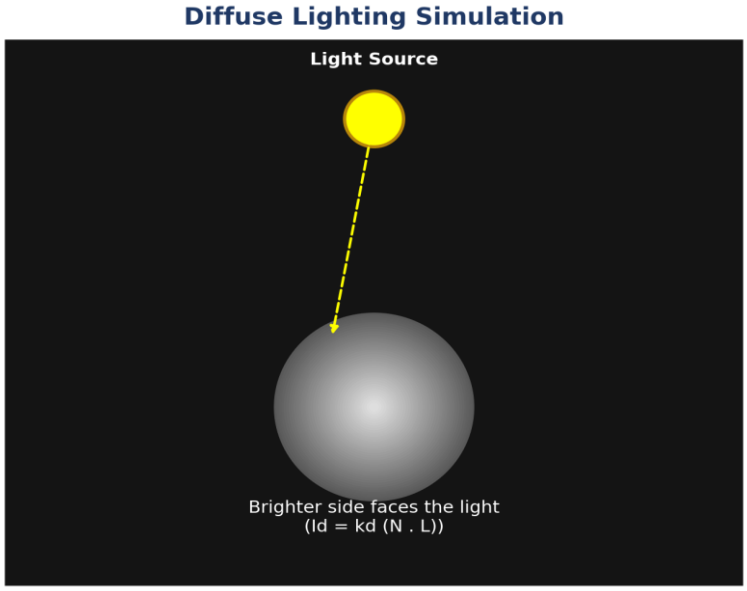


Fig. 5.1 — Diffuse Lighting Concept

Algorithm

1. Import Pygame.
2. Initialize game window.
3. Define light source position.
4. Create a circular object.
5. Calculate distance between light and object.
6. Adjust brightness according to distance.
7. Render the object with varying intensity.
8. Update display continuously.

Program

```
import pygame
import math

pygame.init()

WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Diffuse Lighting Simulation")

clock = pygame.time.Clock()

light_x = 400
light_y = 150
object_x = 400
object_y = 350

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((20, 20, 20))

    # Draw Light Source
    pygame.draw.circle(screen, (255, 255, 0), (light_x, light_y), 20)

    # Distance Calculation
    distance = math.sqrt(
        (object_x - light_x) ** 2 +
        (object_y - light_y) ** 2
    )
    intensity = max(50, 255 - int(distance / 2))
    color = (intensity, intensity, intensity)

    # Draw Object
    pygame.draw.circle(screen, color, (object_x, object_y), 80)
```

```
    pygame.display.update()
    clock.tick(60)

pygame.quit()
```

Explanation

- pygame.draw.circle(screen, (255,255,0), (light_x, light_y), 20) — creates a yellow light source.
- distance = math.sqrt((object_x-light_x)**2 + (object_y-light_y)**2) — calculates the distance between the light source and object.
- intensity = max(50, 255 - int(distance/2)) — as distance increases, brightness decreases, simulating diffuse lighting.

Sample Output

The object appears brighter when closer to the light source and darker when farther away.

Result: Successfully simulated diffuse lighting effects on a game object using Pygame by varying object brightness based on the distance from a light source.

Practical No. 6 : Simulating Specular Lighting Using Pygame

Aim: To simulate specular lighting effects on a game object using Python and Pygame.

Objectives

- Understand the concept of specular reflection.
- Simulate shiny surface effects.
- Learn how highlights are created in computer graphics.
- Implement a simple specular lighting model in Pygame.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code

Theory

Specular lighting produces bright highlights on shiny surfaces such as metal, glass, water, and polished objects. The highlight depends on the positions of the Light Source, the Surface, and the Viewer (Camera). In Direct3D 11, specular lighting is implemented through shaders and is commonly based on the Phong Reflection Model.

The specular lighting equation is:

$$Is = ks (R . V) ^ n$$

- Is — Specular intensity
- ks — Specular reflection coefficient
- R — Reflection vector
- V — View vector
- n — Shininess factor

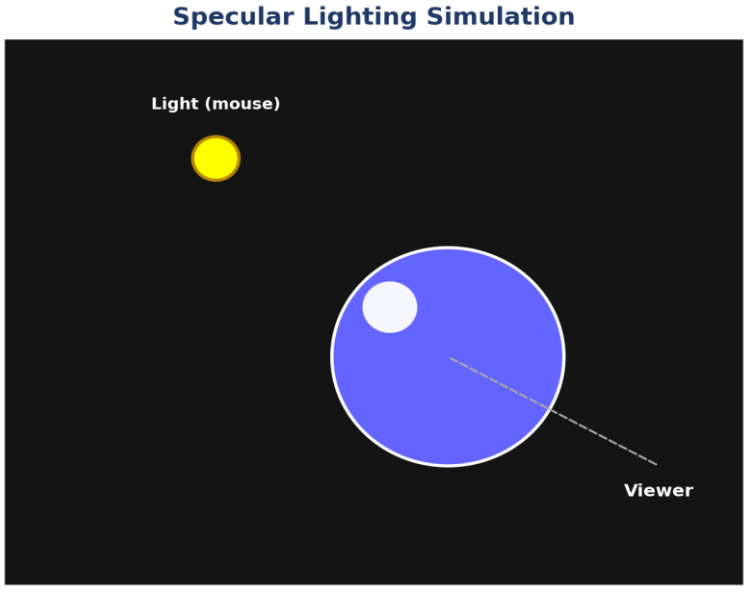


Fig. 6.1 — Specular Reflection Concept

Algorithm

1. Initialize Pygame.

2. Create game window.

3. Define object position.
4. Define movable light source.
5. Draw object using a base color.
6. Draw a small bright spot to represent the specular highlight.
7. Move highlight according to light position.
8. Refresh display continuously.

Program

```
import pygame

pygame.init()

WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Specular Lighting Simulation")

clock = pygame.time.Clock()

light_x = 200
light_y = 150
object_x = 400
object_y = 300

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Move Light Using Mouse
    light_x, light_y = pygame.mouse.get_pos()
    screen.fill((30, 30, 30))

    # Draw Light Source
    pygame.draw.circle(screen, (255, 255, 0), (light_x, light_y), 12)

    # Main Object
    pygame.draw.circle(screen, (100, 100, 255), (object_x, object_y), 100)

    # Specular Highlight
    highlight_x = object_x + (light_x - object_x) // 4
    highlight_y = object_y + (light_y - object_y) // 4
    pygame.draw.circle(screen, (255, 255, 255), (highlight_x, highlight_y), 20)

    pygame.display.update()
    clock.tick(60)

pygame.quit()
```

Explanation

- light_x, light_y = pygame.mouse.get_pos() — moves the light source according to mouse movement.
- pygame.draw.circle(screen, (100,100,255), (object_x,object_y), 100) — creates the object surface.
- pygame.draw.circle(screen, (255,255,255), (highlight_x,highlight_y), 20) — creates a bright white reflection spot that simulates specular lighting.

Sample Output

The yellow circle represents the light source, the blue circle represents the object, and the white spot represents the specular highlight. As the mouse moves, the highlight shifts across the object surface.

Result: Successfully simulated specular lighting by creating a movable light source and displaying a dynamic highlight on the object surface.

Practical No. 7 : Loading and Rendering Game Models Using Pygame

Aim: To load and render a game model (sprite / image) using Python and Pygame.

Objectives

- Understand the concept of game models and sprites.
- Learn image loading and rendering techniques.
- Display graphical game objects on the screen.
- Perform basic object transformation and movement.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code
- A PNG / JPG image file (player.png)

Theory

In DirectX 11, a model is a 3D object composed of vertices, meshes, textures, and materials, loaded from files such as OBJ, FBX, 3DS, or DAE. In Pygame, a model is commonly represented as a sprite (image) — a 2D graphical object used to represent players, enemies, vehicles, and other game elements.

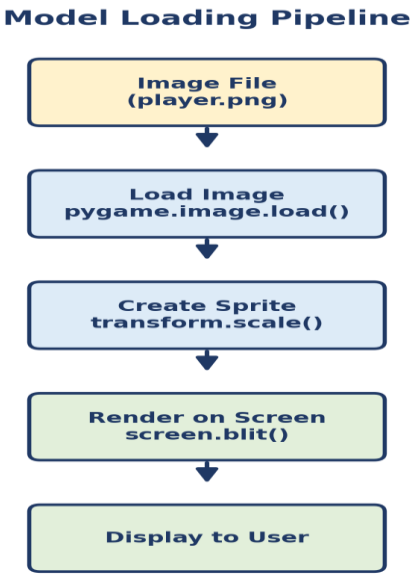


Fig. 7.1 — Model Loading Process

Algorithm

1. Import Pygame library.
2. Initialize Pygame.
3. Create game window.

4. Load model image.
5. Resize image if required.
6. Set initial position.
7. Render image on screen.
8. Move image using keyboard keys.
9. Continuously update display.
10. Exit on close event.

Program

Save an image named "player.png" in the project folder.

```
import pygame

pygame.init()

WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Model Loading and Rendering")

# Load Model (Sprite)
player = pygame.image.load("player.png")

# Resize Sprite
player = pygame.transform.scale(player, (100, 100))

x = 350
y = 250

clock = pygame.time.Clock()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        x -= 5
    if keys[pygame.K_RIGHT]:
        x += 5
    if keys[pygame.K_UP]:
        y -= 5
    if keys[pygame.K_DOWN]:
        y += 5

    screen.fill((30, 30, 30))

    # Render Model
    screen.blit(player, (x, y))
```

```
    pygame.display.update()
    clock.tick(60)

pygame.quit()
```

Explanation

- pygame.image.load("player.png") — loads the image from disk.
- pygame.transform.scale(player, (100,100)) — adjusts the image size.
- screen.blit(player,(x,y)) — draws the sprite on the game window.
- pygame.key.get_pressed() — captures keyboard input for moving the model.

Sample Output

The loaded sprite appears on the screen and can be moved using the arrow keys.

Result: Successfully loaded a game model (sprite image) and rendered it on the screen using Pygame. The model was moved interactively using keyboard controls.

Practical No. 8 : Develop a 2D UFO Catching Game Using Python (Pygame)

Aim: To develop a simple 2D UFO-catching game using Python and Pygame.

Objectives

- Understand 2D game development concepts.
- Implement player movement using keyboard controls.
- Generate random UFO positions.
- Maintain score and game logic.
- Create a complete mini-game.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code

Theory

The Unity 2D UFO Tutorial teaches basic game development concepts such as player movement, object collection, collision detection, and score management. In Python, the same concepts can be implemented using Pygame. When the player touches a UFO, the UFO relocates to a random position and the score increases.

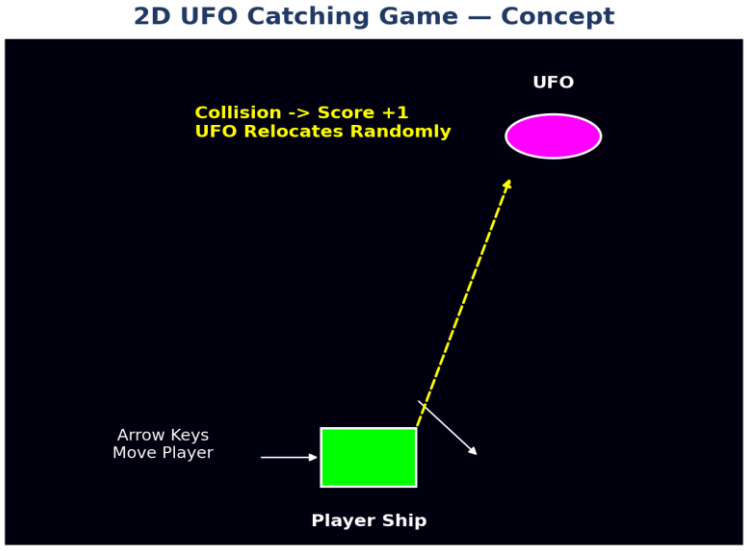


Fig. 8.1 — 2D UFO Catching Game Concept

Algorithm

1. Initialize Pygame.
2. Create game window.
3. Create player object.
4. Generate UFO at a random location.
5. Move player using arrow keys.
6. Detect collision between player and UFO.
7. Increase score on collision.

8. Relocate UFO randomly.
9. Display score continuously.
10. Exit when user closes the game.

Program

```
import pygame
import random

pygame.init()

WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("2D UFO Catching Game")

font = pygame.font.SysFont(None, 40)

player = pygame.Rect(375, 500, 50, 50)
ufo = pygame.Rect(random.randint(50, 750), random.randint(50, 450), 40, 40)

score = 0
clock = pygame.time.Clock()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= 5
    if keys[pygame.K_RIGHT]:
        player.x += 5
    if keys[pygame.K_UP]:
        player.y -= 5
    if keys[pygame.K_DOWN]:
        player.y += 5

    if player.colliderect(ufo):
        score += 1
        ufo.x = random.randint(50, 750)
        ufo.y = random.randint(50, 450)

    screen.fill((0, 0, 20))

    # Player
    pygame.draw.rect(screen, (0, 255, 0), player)

    # UFO
    pygame.draw.ellipse(screen, (255, 0, 255), ufo)
```

```
    score_text = font.render(f"Score : {score}", True, (255, 255, 255))
    screen.blit(score_text, (10, 10))

    pygame.display.update()
    clock.tick(60)

pygame.quit()
```

Explanation

- player = pygame.Rect(375,500,50,50) — creates the player spaceship.
- ufo = pygame.Rect(random.randint(50,750), random.randint(50,450), 40, 40) — creates a UFO at a random location.
- if player.colliderect(ufo): — checks whether the player touches the UFO.
- score += 1 — increases score after collecting a UFO.

Sample Output

A green square represents the player ship, a purple oval represents the UFO, and the score is displayed at the top-left corner.

Result: Successfully developed a 2D UFO-catching game using Pygame with player movement, collision detection, random UFO generation, and score tracking.

Practical No. 9 : Develop a Roll-Ball Game Using Python (Pygame)

Aim: To develop a Roll-Ball game using Python and Pygame, where the player collects objects and increases the score.

Objectives

- Understand object movement and game physics.
- Implement keyboard-based ball movement.
- Create collectible objects.
- Apply collision detection.
- Maintain and display score.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code

Theory

The Unity Roll-a-Ball tutorial is one of the most popular beginner game development tutorials, where the player controls a rolling ball that collects objects placed throughout the game area. In Pygame, the same concept is implemented using a circular player (ball), random collectibles, collision detection, and score tracking.

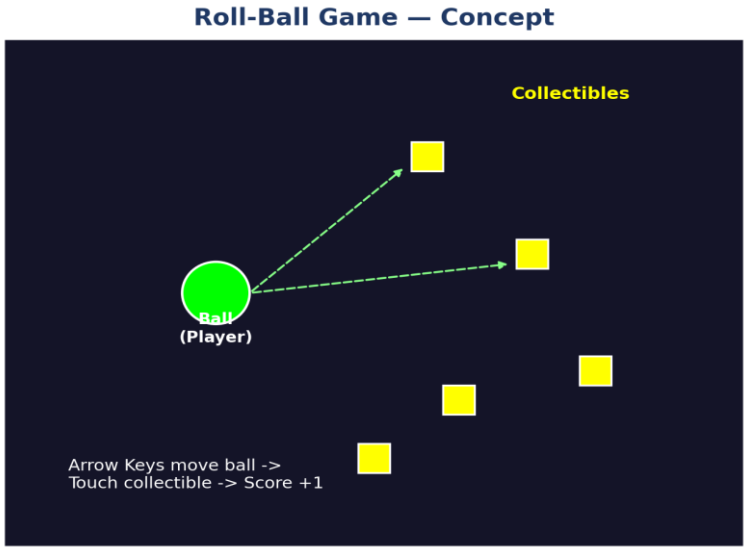


Fig. 9.1 — Roll-Ball Game Concept

Algorithm

1. Initialize Pygame.
2. Create game window.
3. Create a ball object.
4. Generate multiple collectible objects.
5. Move the ball using arrow keys.
6. Detect collision with collectibles.
7. Remove collected objects.
8. Increase score.

9. Display score and remaining collectibles.
10. End game when all collectibles are collected.

Program

```
import pygame
import random

pygame.init()

WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Roll-Ball Game")

font = pygame.font.SysFont(None, 40)

ball_x = 400
ball_y = 300
ball_radius = 20
speed = 5
score = 0

# Create collectibles
collectibles = []
for i in range(10):
    collectibles.append(
        pygame.Rect(random.randint(50, 750), random.randint(50, 550), 20, 20)
    )

clock = pygame.time.Clock()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        ball_x -= speed
    if keys[pygame.K_RIGHT]:
        ball_x += speed
    if keys[pygame.K_UP]:
        ball_y -= speed
    if keys[pygame.K_DOWN]:
        ball_y += speed

    screen.fill((20, 20, 40))

    # Draw Ball
    pygame.draw.circle(screen, (0, 255, 0), (ball_x, ball_y), ball_radius)
```

```
    ball_rect = pygame.Rect(ball_x - ball_radius, ball_y - ball_radius,
                            ball_radius * 2, ball_radius * 2)

    # Draw Collectibles
    for item in collectibles[:]:
        pygame.draw.rect(screen, (255, 255, 0), item)
        if ball_rect.colliderect(item):
            collectibles.remove(item)
            score += 1

    score_text = font.render(f"Score : {score}", True, (255, 255, 255))
    screen.blit(score_text, (10, 10))

    if len(collectibles) == 0:
        win_text = font.render("YOU WIN!", True, (0, 255, 0))
        screen.blit(win_text, (320, 280))

    pygame.display.update()
    clock.tick(60)

pygame.quit()
```

Explanation

- pygame.draw.circle(screen, (0,255,0), (ball_x,ball_y), ball_radius) — creates the player's ball.
- collectibles.append(pygame.Rect(x,y,20,20)) — creates collectible items.
- if ball_rect.colliderect(item): — checks whether the ball touches a collectible.
- score += 1 — increases score when an item is collected.

Sample Output

The rolling ball (green) moves around the screen collecting yellow collectible squares; the score updates as each item is collected.

Result: Successfully developed a Roll-Ball game using Pygame where the player controls a ball, collects objects, and increases the score.

Practical No. 10 : Develop a Space Shooter Game Using Python (Pygame)

Aim: To develop a Space Shooter game using Python and Pygame where the player controls a spaceship, fires bullets, destroys enemies, and scores points.

Objectives

- Understand game object interaction.
- Implement player movement.
- Create a bullet firing mechanism.
- Generate enemy objects.
- Apply collision detection.
- Maintain score and game-over conditions.

Requirements

Software Requirements

- Python
- Pygame
- Visual Studio Code

Theory

The Unity Space Shooter Tutorial introduces concepts such as player spaceship control, enemy spawning, bullet firing, collision detection, score management, and game over conditions. In Pygame, these concepts are implemented using simple shapes and event handling.

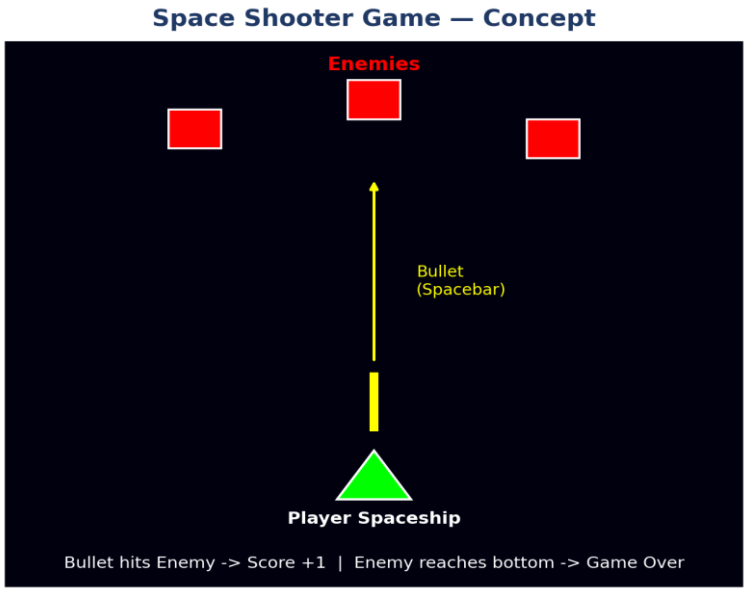


Fig. 10.1 — Space Shooter Game Concept

Algorithm

1. Initialize Pygame.
2. Create game window.
3. Create spaceship object.
4. Move spaceship using arrow keys.

5. Fire bullets using the Spacebar.
6. Generate enemies at random positions.
7. Move enemies downward.
8. Detect bullet-enemy collisions.
9. Increase score when an enemy is destroyed.
10. Display Game Over if an enemy reaches the bottom.
11. Exit when the user closes the game.

Program

```
import pygame
import random

pygame.init()

WIDTH = 800
HEIGHT = 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Space Shooter Game")

font = pygame.font.SysFont(None, 40)

# Player
player = pygame.Rect(375, 520, 50, 50)

# Bullets
bullets = []

# Enemies
enemies = []
for i in range(5):
    enemies.append(pygame.Rect(random.randint(50, 750), random.randint(20, 150), 40, 40))

score = 0
game_over = False
clock = pygame.time.Clock()
running = True

while running:
    clock.tick(60)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE:
                bullets.append(pygame.Rect(player.x + 22, player.y, 6, 15))

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= 6
    if keys[pygame.K_RIGHT]:
```

```
    player.x += 6
player.x = max(0, min(player.x, WIDTH - player.width))

if not game_over:
    # Move bullets
    for bullet in bullets[:]:
        bullet.y -= 8
        if bullet.y < 0:
            bullets.remove(bullet)

    # Move enemies
    for enemy in enemies[:]:
        enemy.y += 2
        if enemy.y > HEIGHT:
            game_over = True

    # Collision detection
    for bullet in bullets[:]:
        for enemy in enemies[:]:
            if bullet.colliderect(enemy):
                if bullet in bullets:
                    bullets.remove(bullet)
                if enemy in enemies:
                    enemies.remove(enemy)
                score += 1
                enemies.append(pygame.Rect(random.randint(50, 750), 0, 40, 40))
                break

# Drawing Section
screen.fill((0, 0, 30))

# Player spaceship
pygame.draw.polygon(screen, (0, 255, 0), [
    (player.x + 25, player.y),
    (player.x, player.y + 50),
    (player.x + 50, player.y + 50)
])

# Bullets
for bullet in bullets:
    pygame.draw.rect(screen, (255, 255, 0), bullet)

# Enemies
for enemy in enemies:
    pygame.draw.rect(screen, (255, 0, 0), enemy)

score_text = font.render(f"Score : {score}", True, (255, 255, 255))
screen.blit(score_text, (10, 10))

if game_over:
```

```
        over_text = font.render("GAME OVER", True, (255, 0, 0))
        screen.blit(over_text, (300, 280))

    pygame.display.update()

pygame.quit()
```

Explanation

- player = pygame.Rect(375,520,50,50) — creates the player-controlled spaceship.
- if event.key == pygame.K_SPACE: — fires a bullet when the Spacebar is pressed.
- random.randint(50,750) — creates enemies at random positions.
- bullet.colliderect(enemy) — checks if a bullet hits an enemy.
- score += 1 — updates the score after destroying an enemy.

Sample Output

The player spaceship (green) fires bullets (yellow) upward to destroy enemies (red); the score is displayed at the top-left, and “GAME OVER” is shown if an enemy reaches the bottom of the screen.

Result: Successfully developed a Space Shooter game using Pygame with player movement, bullet firing, enemy spawning, collision detection, scoring, and game-over functionality.